

Sensitivity-Aware Placement Sustains All-Analog GPT-2 Inference on Aging In-Memory Tiles

Wei Ju Hwang

Abstract—Analog in-memory computing (AIMC) tiles are heterogeneous at fabrication and degrade unevenly in the field, yet the assignment of weight blocks to physical tiles (*placement*) is free at map time. We study whether this decision determines deployment quality for a decoder-only language model executed *entirely* on analog tiles. A five-phase pipeline combines hardware-aware fine-tuning of GPT-2, a measured per-projection sensitivity profile, and static shard placement onto a simulated 96×8 substrate that drifts, fluctuates, and suffers faults over a 120-step aging trace. On the full WikiText-103 test set, across five independent hardware traces, sensitivity-aware placement improves NLL over the strongest workload-blind baseline by 0.063 nats averaged over the simulated lifetime (95% CI [0.041, 0.084]) and by 0.085 nats at end of life, holding the end-of-life penalty relative to the pretrained digital model to 0.30 nats where workload-blind policies reach 0.39–0.45 and keep growing. The one-shot placement is statistically indistinguishable from a clairvoyant placement that knows the entire degradation trajectory, and a six-scenario sweep finds the benefit monotone in tile-quality spread. Placement, not just noise tolerance, decides how fast an all-analog LLM deployment ages.

Index Terms—Analog in-memory computing, placement, large language models, hardware-aware training, reliability.

I. INTRODUCTION

ANALOG in-memory computing executes matrix-vector products inside memory crossbars, promising large energy and latency gains for DNN inference at the cost of storing weights as noisy conductances [1], [2]. Real substrates are *heterogeneous* and *non-stationary*: programming-noise spread across tiles, conductance drift, spatially correlated thermal variation, and localized permanent faults are all documented at scale [1], [3], [4]. A deployment must therefore decide which weight block lives on which physical tile, and tiles differ in quality and age differently.

Existing heterogeneous-mapping work optimizes a different axis: which operations stay *digital*. Layer-, channel-, and weight-granular methods partition workloads across analog tiles and digital compute units [10], [11], leaving the intra-analog assignment unspecified; in practice it defaults to the sequential or random orders we use as baselines. Fault-aware remapping *within* a crossbar [12], [13] is CNN-scale and single-snapshot, often with retraining; tile-granular quality differences of an aging substrate

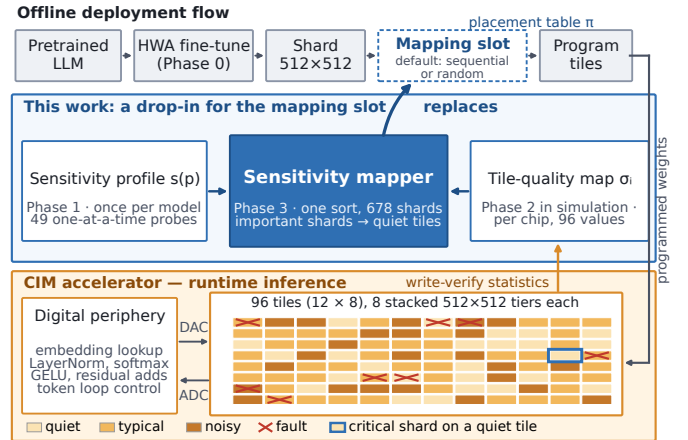


Fig. 1. The sensitivity mapper replaces the toolchain’s mapping slot in an LLM-on-CIM deployment, consuming a one-off model sensitivity profile and a per-chip tile-quality map and emitting the shard-to-tile placement table; no additional runtime hardware. All 96 tiles are drawn, shaded by noise class.

remain unaddressed. A recent letter distills the partitioning methods into a unified workflow and contributes the first AIMC precision-sensitivity profile of a decoder-only transformer [9], leaving two questions open: whether time-dependent degradation invalidates static mappings, and how mapping should exploit intra-analog tile-quality differences. We answer both for a fully analog deployment: *no digital fallback*, placement the only degree of freedom.

Contributions. (i) An end-to-end pipeline that makes pretrained GPT-2 functional under a documented analog noise contract and evaluates placement on the full WikiText-103 test set under time-resolved aging; (ii) a paired, five-trace statistical design with identical noise co-ordinate fields across policies; (iii) evidence that placement controls how fast the deployment ages (end-of-life penalty relative to the pretrained digital model 0.30 nats, versus 0.39–0.45 for every workload-blind policy); (iv) adversarial and clairvoyant placement bounds showing the one-shot policy is near-optimal.

II. ALL-ANALOG DEPLOYMENT PIPELINE

The pipeline has five phases sharing one frozen configuration and one noise contract; only the hardware-trace seed varies between runs. All experiments use GPT-2 Small (124M) [5] and WikiText-103 [8] (train split for fine-tuning, validation for calibration, held-out test for all reported quality numbers), simulated with IBM’s AIHWKit [6], [7]. Fig. 1 situates the contribution; in deployment, Phase 2’s role is played by per-chip calibration.

A. Noise Contract

Every analog projection is clipped once at 2.5σ of its weight distribution (symmetric about zero, as for differential conductance pairs), tiled into 512×512 crossbar shards, and converted with ~ 8 -bit ADC/DAC. Additive Gaussian weight noise is expressed as a normalized standard deviation, a fraction of the projection's programmed (peak-to-peak) range R_p , with deployment reference $\sigma_{\text{ref}}=0.023$, calibrated to measured PCM programming noise [1], [7]: $0.023R_p$ equals the RMS quantization error ($\Delta/\sqrt{12}$, $\Delta = R_p/2^b$) of a $b \approx 3.7$ -bit uniform quantizer, the sense in which we quote ≈ 4 -bit effective precision. AIHWKit's internal noise sources are disabled, so this contract is the *only* noise in the simulation; it coincides setting-for-setting with IBM's GPT-2 AIMC study [9], chosen independently from the same measurements.

B. Phase 0: Hardware-Aware Fine-Tuning

Post-training conversion of pretrained GPT-2 to this contract is non-functional (perplexity $29 \rightarrow \approx 3,146$ before any tile noise). We therefore fine-tune with perturb-forward/clean-update noise injection [1], [2]: each forward pass clips via a straight-through estimator and perturbs every projection with $\sigma \sim \mathcal{U}(0, 2\sigma_{\text{ref}})$ (2,000 steps, AdamW, learning rate 5×10^{-5} with cosine decay and 100 warmup steps, weight decay 0.01, batch size 8 with 4-step gradient accumulation, sequence length 512, bf16). The tied LM head is noised only in its matmul role, exactly as deployed. The training envelope covers every tile at map time (class multipliers $0.65\text{--}1.70\times$) but not the late-life extremes: drift and faults push 5–12 of the 96 tiles above $2\sigma_{\text{ref}}$ by end of life (peak $3.1\sigma_{\text{ref}}$ across the five traces), so the deployment is evaluated beyond its training envelope; Section IV-B quantifies how much importance each policy leaves there.

C. Phase 1: Measured Sensitivity Profile

For each of the 49 projections p (48 transformer projections + LM head), one projection at a time is made analog while the rest stay digital, and

$$s(p) = \mathbb{E}_a[\text{NLL}(W_p^c + \sigma_{\text{ref}}R_pZ_a)] - \text{NLL}_{\text{ref}}(p), \quad (1)$$

with five antithetic noise pairs Z_a and $\text{NLL}_{\text{ref}}(p)$ that projection's clipped, quantized, zero-noise reference (isolating noise from conversion effects). The profile is additive by construction (each projection is measured in isolation), an assumption Section V revisits. It is strongly structured (Fig. 2): the LM head (0.061 nats) and block 0's attention output (0.034) dominate, with a rank-1-to-rank-49 spread of $\sim 540\times$; the ordering matches the pretrained model's profile at $\sim 100\times$ larger magnitudes, so the structure is architectural and survives hardware-aware training, corroborating [9]. Weight magnitude, the obvious free proxy, is uncorrelated with it (Spearman $\rho = -0.08$). The profile costs 540 forward passes over the 250k-token calibration split, once per model, not per chip.

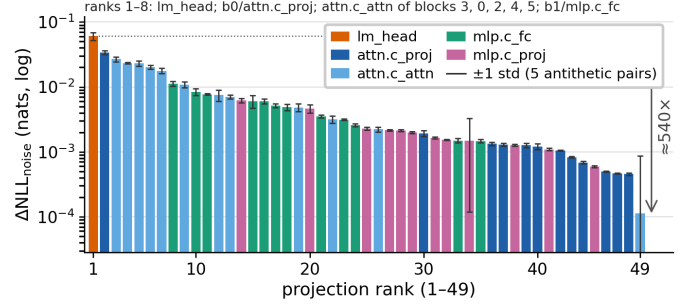


Fig. 2. Measured post-HWA sensitivity (one profiling seed) of all 49 GPT-2 projections at $\sigma_{\text{ref}}=0.023$, colored by projection type (the eight most sensitive listed by block and type); error bars: ± 1 std over the five antithetic noise pairs. The LM head and early-block attention dominate by orders of magnitude; the rank-49 tail sits at the measurement floor.

D. Phase 2: Time-Varying Tile Fidelity

The substrate has 96 tiles, each carrying 8 *tiers*: vertically stacked 512×512 crossbars that share the tile's periphery. Tile i 's normalized noise at timestep $t \in [0, 119]$ is

$$\sigma_i(t) = b_i (d_i(t) + H_{z(i)}(t)) \cdot f_i(t), \quad (2)$$

where $b_i = \sigma_{\text{ref}}m_i$ with class multipliers m_i drawn from a 25/50/25% healthy/typical/poor taxonomy ($0.65\text{--}1.70\times$, spanning reported array-to-array spread [1]); $d_i(t)$ ramps $+12\text{--}35\%$ over the window (PCM drift exponents $\nu \approx 0.03\text{--}0.1$ [3]); H_z is zone-correlated AR(1) variation ($\rho = 0.94$, 8 zones) [4]; and $f_i(t)$ steps permanently from 1 to $1.25\text{--}1.70$ on 8 tiles at onsets $t \sim \mathcal{U}(35, 90)$ [4]. All eight tiers of a tile share $\sigma_i(t)$: the quality map offers 96 distinct values for 678 placement decisions, so the mapper consumes per-tile write-verify statistics, not per-crossbar characterization. Values are scenario parameters motivated by these measurements, not calibrated to one device; timesteps are dimensionless; Section IV-D varies them.

E. Phase 3: Sharding and Placement Policies

The 49 projections tile into 678 shards (fused Q/K/V region boundaries are never crossed); a placement is a map π from shards to the 768 physical tiers. Shard importance is $I_s = \max(s(p), 0) |W_s|/|W_p|$, and placements are compared at map time by the separable diagnostic $\Phi = \sum_s I_s \sigma_{\pi(s)}^2$. Policies: **random** (seeded shuffle); **sequential** (slot order); **hardware_only** (quietest tiles first, shard order random—equivalently, random placement restricted to the quietest 678 tiers: hardware-aware but workload-blind, the strongest baseline); and **static_sensitivity** (quietest tiles first, most important shards first), which is provably Φ -optimal at $t=0$ by the rearrangement inequality. The 90 spare tiers let quality-sorted policies leave the worst capacity empty; **hardware_only** shares this advantage, so its gap isolates importance ordering. The mapper is one sort over the 678 shards. Two bounds complete the span: **adversarial** (most important shards onto the noisiest tiles) and **oracle_clairvoyant**, the same sorted rule applied to the *trajectory-RMS* noise

$\bar{\sigma}_i = (\frac{1}{T} \sum_t \sigma_i^2(t))^{1/2}$ —a reference that knows every future fault and drift rate at map time; all placements are static.

III. PHASE 4: LIFETIME EVALUATION PROTOCOL

For each policy, timestep $t \in \{0, 30, 60, 90, 119\}$, and realization $r \in \{1, 2, 3\}$, tile noise is materialized once into the logical weights, $W^{\text{noisy}}[s] = W^c[s] + \sigma_{\pi(s)}(t) R_p Z_{p,r}[s]$, and the full test set (285,417 predicted tokens) is evaluated through the AIHWKit analog forward path. The standard-normal field $Z_{p,r}$ is keyed by projection and realization only, so *policies see identical noise coordinates and differ solely through tile scales*: paired differences are free of realization variance (seeded policy orders vary per trace). Three references frame the results: the *pretrained digital* model (PPL = 29.0, NLL 3.368 on the same windows), the alternative a practitioner would deploy; the *nominal* all-analog model (clip + quantization, zero tile noise; 31.50); and the HWA model’s own *unclipped digital forward* (42.91), a diagnostic only (Section IV-A). Quality is token-weighted NLL in nats per token (hereafter nats); perplexity (e^{NLL}) is reported for intuition, since paired ΔNLL differences are level-independent. The 15 within-trace samples share one correlated trajectory, so *the five independent traces (seeds 41–45) are the statistical units*; intervals are Student- t 95% CIs with $n=5$ (hereafter 95% CI).

IV. RESULTS

A. The Deployment Is Functional; Placement Sets Its Cost

Phase 0 eliminates the post-training conversion collapse ($29 \rightarrow 3,146$): the nominal all-analog model sits 0.082 nats above the pretrained digital reference (31.50 versus 29.0 PPL). The HWA model evaluated *without* the 2.5σ clip it was trained through is off-distribution and lands at 42.91, a diagnostic only, not a deployment option. Under the aging trace (Fig. 3a), placement decides how the 0.082-nat penalty grows. With sensitivity-aware placement the deployment starts at ≈ 36 PPL and ends at 39.30 (five-trace mean), an end-of-life penalty of 0.303 nats; **hardware_only** reaches 0.389, and **random** and **sequential** 0.447 and 0.448 (Table I), still rising. Sensitivity-aware placement is also the only policy that stays below even the diagnostic reference in all 75 lifetime conditions (smallest margin ≈ 2.6 PPL), whereas **random** and **sequential** cross it at end of life and **hardware_only** does in 6/15 conditions (Table I). Whether 0.30 nats is worth the analog gains is a workload question; what placement controls is the growth.

B. Placement Wins, and the Edge Grows with Age

The gap columns of Table II report the paired improvement of sensitivity-aware placement over each baseline, averaged over each trace’s 15 lifetime conditions, with the five traces as units. Every CI excludes zero, all five traces and all 75 paired conditions favor sensitivity-aware placement against every baseline, and the weakest trace still improves by +0.046 nats over the strongest baseline.

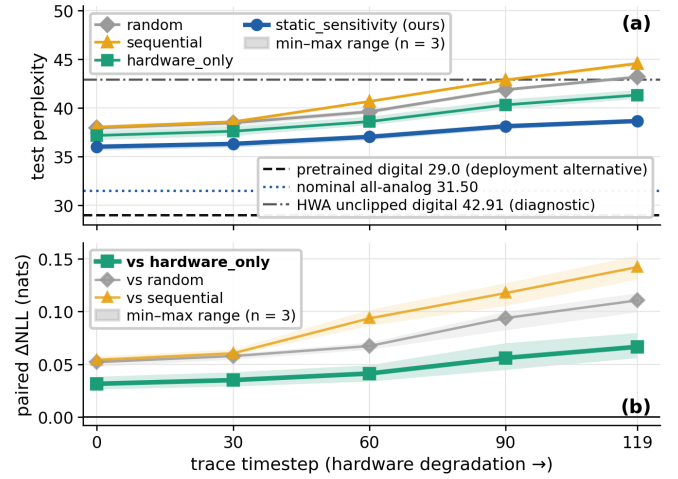


Fig. 3. Trace 42, the weakest of the five traces by lifetime gain over **hardware_only**; bands: min-max range over the three paired realizations ($n=3$). (a) Held-out test perplexity; reference lines: pretrained digital (deployment alternative), nominal all-analog, HWA unclipped digital forward (diagnostic). (b) Paired improvement of sensitivity-aware placement per timestep; the advantage grows monotonically with degradation.

TABLE I
END-OF-LIFE ($t=119$) TEST PERPLEXITY OVER 15 CONDITIONS (5 TRACES $\times$ 3 REALIZATIONS): “MEAN”/“WORST”, MEAN AND MAXIMUM PPL OVER THE 15 CONDITIONS; “STD”, ACROSS THE FIVE TRACE MEANS; Δ_{29} , END-OF-LIFE NLL PENALTY VERSUS THE PRETRAINED MODEL (NATS); “>DIAG.”, CONDITIONS ABOVE THE 42.91 DIAGNOSTIC.

Policy	mean	std	worst	Δ_{29}	>diag.
sequential	45.48	2.75	50.43	0.448	13/15
random	45.38	1.84	48.05	0.447	15/15
hardware_only	42.82	1.61	45.90	0.389	6/15
static_sensitivity	39.30	0.60	40.28	0.303	0/15

The end-of-life effect is larger: +0.085 nats ($[0.044, 0.127]$) against **hardware_only**, +0.143 and +0.145 against **random** and **sequential**. On the per-timestep trace (Fig. 3b) the advantage over **hardware_only** doubles from 0.031 to 0.067 nats between $t=0$ and $t=119$, monotonically on this trace and in the five-trace mean (individual traces fluctuate): sensitivity-aware placement *ages better*, because the shards that matter most sit on the tiles that degrade least. The placements show this directly: at end of life, sensitivity-aware placement leaves 1.0% of shard importance on tiles above the $2\sigma_{\text{ref}}$ training envelope, versus 5.2% (**hardware_only**), 9.6–10.2% (**random**, **sequential**) and 33% (**adversarial**); its importance-weighted tile noise is $1.06\sigma_{\text{ref}}$ versus 1.30 and 1.38–1.39. Hardware awareness alone recovers about 40% of the benefit over **random**; knowing *which shard matters* recovers the rest. Placement also de-risks: end-of-life trace-to-trace spread is 2.7–4.6 $\times$ smaller than the baselines’ (Table I).

C. One-Shot Placement Is Near-Oracle

The two bounding placements of Table II, evaluated on the same five traces with exactly paired noise, complete the span. Sensitivity-aware placement captures 97.8% of the headroom available from trajectory knowledge (random-to-clairvoyant span; 96.3% measured from

TABLE II

PLACEMENT SPAN AND PAIRED GAPS: MEAN $\Delta\text{NLL}_{\text{tile}}$ (NATS, DEGRADATION RELATIVE TO NOMINAL) OVER ALL TIMESTEPS, REALIZATIONS, AND FIVE TRACES; “GAP”: PAIRED ΔNLL OF EACH PLACEMENT MINUS THE PROPOSED POLICY PER TRACE (5 TIMESTEPS $\times$ 3 REALIZATIONS), FIVE TRACES AS UNITS, 95% CI (t , $n=5$); EVERY BASELINE GAP IS POSITIVE IN 5/5 TRACES.

Placement	mean	gap	95% CI	role
adversarial	0.501	+0.328	[0.268, 0.387]	worst case
random	0.281	+0.107	[0.067, 0.147]	baseline
sequential	0.280	+0.106	[0.048, 0.164]	baseline
hardware_only	0.236	+0.063	[0.041, 0.084]	strongest baseline
static_sensitivity	0.173	—	—	proposed
oracle_clairvoyant	0.171	−0.002	[−0.008, 0.003]	clairvoyant ref.

hardware_only, 99.3% of the adversarial-to-clairvoyant span; headroom from a better importance model lies outside all three); its residual gap to the clairvoyant reference, 0.002 nats (95% CI [−0.003, 0.008] across traces), is at most an eighth of the headline effect and *statistically indistinguishable from zero*. Knowing the future buys essentially nothing, bounding the value of adaptive remapping: importance weighting already parks only low-importance shards on fault-prone capacity, so faults arrive pre-mitigated. (The clairvoyant is Φ -optimal, not NLL-optimal: a *reference*, not a bound.) Two further readings: **sequential** (0.280) and **random** (0.281) are indistinguishable, so a toolchain’s default slot order is no better than chance with respect to tile quality; and the adversarial edge sits $1.8\times$ as far from nominal as **random**, above even the diagnostic reference already at $t=0$ (≈ 45 PPL; 55–85 by end of life). Placement alone spans the range between a usable deployment and an unusable one.

D. Robustness Across Hardware Scenarios

Because the Phase-2 parameters are scenario values, we vary them one factor at a time around the frozen configuration (Table III; one trace each, $t \in \{0, 119\} \times 2$ realizations). The paired improvement is positive in **48/48** conditions (6 variants $\times$ 2 baselines $\times$ 2 timesteps $\times$ 2 realizations) and scales with tile-quality spread: +0.027 (mild), +0.050 (default, same protocol) and +0.073 nats (harsh) against the strongest baseline, a $2.7\times$ swing. Sensitivity-aware placement ends at 38.7–41.7 PPL (penalty 0.29–0.36 nats versus the pretrained model, worst under doubled drift), below the diagnostic reference in all six variants, whereas the baselines’ mean end-of-life PPL crosses above it in the harsher half (**random** in three variants, **hardware_only** in two). The less uniform the substrate, the more placement pays.

V. DISCUSSION AND LIMITATIONS

For a fully analog decoder LLM, this answers the two questions left open in [9]: static mappings *are* invalidated by degradation, but only workload-blind ones, and intra-analog tile-quality differences are the resource placement exploits.

Deliberate boundaries: simulation only, under a documented contract; tile-level noise; fixed substrate geometry

TABLE III

SCENARIO SWEEP (ONE TRACE, $t \in \{0, 119\} \times 2$ REALIZATIONS): PAIRED IMPROVEMENT OF SENSITIVITY-AWARE PLACEMENT (NATS) AND ITS END-OF-LIFE PPL; DIAGNOSTIC = 42.91.

Variant	vs hw_only	vs random	EOL PPL
spread_low (0.8–1.3 $\times$)	+0.027	+0.043	39.80
faults_low (4 tiles)	+0.042	+0.073	38.83
thermal_low ($\rho=0.7$)	+0.052	+0.075	38.71
faults_high (16 tiles)	+0.066	+0.095	39.56
drift_high (0.25–0.60)	+0.068	+0.106	41.69
spread_high (0.5–2.2 $\times$)	+0.073	+0.123	38.85

(tile/tier counts, 90 spare tiers); no hybrid baseline (the LM head kept digital is the natural next comparison); additive per-projection sensitivity, measured one projection at a time while deployment noises all 49 at once (the leave-one-out profile that tests rank stability is implemented in the released code but not yet run); uniform within-projection sensitivity; dimensionless timesteps, so claims concern policy ordering and mechanism, not calendar lifetime; energy and latency not modeled. Scale generality is a hypothesis, not a result: the LM head’s parameter share falls from $\approx 30\%$ at 124M to a few percent at multi-billion scale, so the sensitivity concentration driving the effect may weaken with size; the prepared GPT-2 Medium configuration is the first test. The tile-quality map is the write-verify statistic gathered during programming, so the mapper’s inputs exist in current deployment flows.

Reproducibility: code, configurations, and figure data: <https://github.com/williamhwangweiju/projection-sensitivity-mapping>.

VI. CONCLUSION

For an all-analog GPT-2 on heterogeneous, aging in-memory tiles, placement decides how fast the deployment ages. A measured projection-sensitivity profile turns this free map-time choice into most of what a clairvoyant mapper could achieve: +0.063 nats over the strongest baseline averaged over the simulated lifetime (5/5 traces), +0.085 nats at end of life, and an end-of-life penalty of 0.30 nats versus the pretrained digital model where workload-blind policies reach 0.39–0.45.

REFERENCES

- [1] V. Joshi *et al.*, “Accurate deep neural network inference using computational phase-change memory,” *Nat. Commun.*, vol. 11, 2020.
- [2] M. J. Rasch *et al.*, “Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators,” *Nat. Commun.*, vol. 14, 2023.
- [3] M. Le Gallo *et al.*, “Mixed-precision in-memory computing,” *Nat. Electron.*, vol. 1, 2018.
- [4] W. Wan *et al.*, “A compute-in-memory chip based on resistive random-access memory,” *Nature*, vol. 608, 2022.
- [5] A. Radford *et al.*, “Language models are unsupervised multitask learners,” OpenAI, 2019.
- [6] M. J. Rasch *et al.*, “A flexible and fast PyTorch toolkit for simulating training and inference on analog crossbar arrays,” in *Proc. IEEE AICAS*, 2021.
- [7] M. Le Gallo *et al.*, “Using the IBM analog in-memory hardware acceleration kit for neural network training and inference,” *APL Mach. Learn.*, vol. 1, 2023.
- [8] S. Merity *et al.*, “Pointer sentinel mixture models,” in *Proc. ICLR*, 2017.

- [9] C. Lammie, “Heterogeneous mapping for analog in-memory computing accelerators: A unified workflow,” *IEEE Comput. Archit. Lett.*, 2026.
- [10] C. Lammie *et al.*, “LionHeart: A layer-based mapping framework for heterogeneous systems with analog in-memory computing tiles,” *IEEE Trans. Emerg. Topics Comput.*, 2025.
- [11] H. Benmeziiane *et al.*, “Supernetwork-based efficient mapping of deep learning applications to mixed-precision hardware using model adaptation,” *Nat. Commun.*, 2026.
- [12] L. Xia *et al.*, “Stuck-at fault tolerance in RRAM computing systems,” *IEEE JETCAS*, vol. 8, no. 1, 2018.
- [13] B. Zhang *et al.*, “Handling stuck-at-faults in memristor crossbar arrays using matrix transformations,” in *Proc. ASP-DAC*, 2019.